

Problem 1:

To generate inputs, I prepended each ASCII digit '0' through '9' to my Stevens ID (soh) to form a prefix pre.

I then hashed strings of the form pre + counter (where the counter is a 4 byte little endian integer starting at 0) and stored each digest in a dictionary.

As soon as a digest appeared for the second time, a collision was found and I recorded the total number of evaluations.

The birthday paradox states that collisions in a hash function with n bit output should appear after approximately $2^{(n/2)}$ evaluations.

Since proj2hash outputs 5 bytes = 40 bits, the expected number of evaluations is $2^{20} \approx 1,048,576$.

Across 10 trials, my observed average was 1,289,670 evaluations, which is within about 23% of the theoretical prediction.

This is consistent with the birthday bound where the small overshoot is expected due to random variance across trials.

Problem 2:

To generate inputs with the \$100 meaning, I combined multiple independent tweaks that preserve semantics: prefix words ('', Note:, Reminder:, FYI:), name capitalization (Alex, ALEX, alex), verb form (owes, owes you, currently owes), four independent single vs double spacing positions, amount spelling (100, one hundred, 100.00, a hundred, \$100), currency unit (dollars, USD, bucks, ...), timeframe phrasing (5 variants of "by tomorrow"), a confirmation phrase, punctuation, and a closing line.

Combining these dimensions yields approximately 2.6 million distinct \$100 variants.

An analogous set was built for the \$1,000,000 meaning using 6 amount spellings (1,000,000, one million, 1M, ...), yielding about 3.1 million variants.

I hashed all \$100 variants into a dictionary, then streamed through \$1,000,000 variants until a digest matched.

In practice, a collision was found after 3,173,718 total evaluations (2,591,998 for the \$100 table and 581,720 \$1M candidates tried before a match).

The collision found was:

- M1: ALEX: Reminder: ALEX owes soh 100.00 dollars US by end of tomorrow per our agreement Thanks.
- M2: alex owes soh 1 million dollars by end of day per our agreement! Please confirm receipt.

Both hash to: f819dccf79

Problem 3:

To generate inputs, each trial uses a 5 byte seed derived by hashing pre = (digit || id).

The sequence x_0 , $f(x_0)$, $f^2(x_0)$, ... with $f = \text{proj2hash}$ then lives entirely in the 5 byte output space with no additional input choices

needed.

Floyd's algorithm has two loops.

Phase 1 (detection): the tortoise advances one step per iteration (1 eval) and the hare advances two (2 evals), for 3 evaluations per iteration, until they meet inside the cycle.

Phase 2 (location): both tortoise (reset to x_0) and hare advance one step each (2 evals per iteration) until they meet at the start of the cycle, at which point the previous values form the colliding pair.

In lecture, a random walk on 2^n values collides with itself after $\sim 2^{(n/2)}$ steps, where $n = 40$ and $2^{20} \approx 1,048,576$.

Floyd's Phase 1 runs for $\sim 2^{20}$ iterations $\times$ 3 evals ~ 3.1 M evals, and Phase 2 adds $\sim 2^{20} \times 2$ evals ~ 2.1 M, for a theoretical total of $\sim 5 - 7$ M.

My observed average was 4,313,362 evaluations which is about 4 times the birthday bound of 2^{20} , consistent with this prediction.

The increased cost over Problem 1 is the price of using $O(1)$ memory instead of a full dictionary.

Problem 4:

To generate inputs, I defined an injective function f mapping every 5 byte hash output to a unique meaningful message.

Injectivity is essential since if f is many to one, Floyd's almost always finds a trivial collision where $f(x) = f(y)$ despite x not equalling y , which is not a proj2hash collision at all.

The design encodes the top bit of the input as the semantic meaning (0 $\rightarrow$ \$100, 1 $\rightarrow$ \$1M), the next few bits select readable variants (name capitalization, amount spelling), and the remaining ~ 35 bits are embedded as a unique transaction reference number [ref: N] in the message.

This guarantees that every distinct 5 byte input maps to a distinct message, so any collision $H'(x) = H'(y)$ with x not equal to y is automatically a genuine proj2hash collision.

A collision is not useful when both x and y share the same top bit. So when both messages carry the same semantic meaning (\$100 or \$1M). Since the top bit is essentially random, each trial is useful with probability $1/2$.

The expected number of restarts is therefore 2.

Which is the number of trials of a geometric distribution with $p = 1/2$.

In my test run, a useful collision was found on attempt 4 (total evaluations across all attempts: 9,851,162):

– M1 (\$1M): Alex owes soh 1,000,000 dollars by tomorrow. [ref: 24192040478]

– M2 (\$100): alex owes soh one hundred dollars by tomorrow. [ref: 82028837226]

Both hash to: 0bc3eca371